Functional Programming — Assignment 5

Student Name: Maxsutkhan Ramazanov | **Group:** IT-2506 | **Week:** 5
Assignment Title: Composing Functions Using Pipelining

1. Introduction and Problem Context

In e-commerce pricing engines, order totals are calculated through a series of discrete pricing rules: applying promotional discounts, adding delivery or service surcharges, and rounding to currency decimals.

In conventional imperative code, these steps are frequently executed as deeply nested function invocations:

```
var price = RoundToCents(AddFee(5m, ApplyDiscount(0.10m, 1234.56m)));
```

Problems with Nested Invocations

- **Inside-Out Evaluation:** Readers must parse the expression from the deepest parenthesis outward, which opposes the natural chronological flow of data processing.
- **Rigid Structure:** Adding, removing, or reordering steps requires manual rewriting of nested parentheses.
- **Low Reusability:** Rules cannot easily be extracted, configured per customer tier, or stored in collections dynamically.

Assignment 5 refactors this calculation using foundational functional programming patterns: Currying, Partial Application, Function Composition (`Compose`), Fluent Extension Pipelining (`Pipe`), and Fold-based Dynamic Pipelines (`Pipeline` via `Aggregate`).

2. Implementation Tasks

2.1 Task 1: Currying and Partial Application

Currying transforms a multi-argument function into a chain of unary functions: (A × B → C) ⇒ (A → (B → C)).

```
public static Func<A, Func<B, C>> Curry<A, B, C>(Func<A, B, C> f)
{
```

```
    return a => b => f(a, b);
}
```

Using `Curry`, binary operations `ApplyDiscount(rate, amount)` and `AddFee(fee, amount)` are curried. Passing configuration values (`rate = 0.10m`, `fee = 5m`) produces unary functions of type `Func<decimal, decimal>` via partial application:

```
var curriedDiscount = Curry<decimal, decimal, decimal>(ApplyDiscount);
var curriedFee = Curry<decimal, decimal, decimal>(AddFee);

Func<decimal, decimal> discount10 = curriedDiscount(0.10m);
Func<decimal, decimal> fee5 = curriedFee(5m);
```

2.2 Task 2: Function Composition (`Compose`)

Forward composition chains two functions such that the output of *f* becomes the input to *g*: (f ⋙ g)(x) = g(f(x)).

```
public static Func<A, C> Compose<A, B, C>(Func<A, B> f, Func<B, C> g)
{
    return x => g(f(x));
}
```

A complete reusable pricing function `checkout` is built by composing `discount10`, `fee5`, and `RoundToCents`:

```
Func<decimal, decimal> checkout = Compose(Compose(discount10, fee5), round
```

For input `1234.56m`, evaluating `checkout(1234.56m)` produces `1116.10`.

2.3 Task 3: Fluent Pipelining Extension (`Pipe`)

To achieve left-to-right syntax directly on values, an extension method is defined:

```
public static class PipeExtensions
{
    public static R Pipe<T, R>(this T value, Func<T, R> f)
    {
        return f(value);
    }
}
```

This transforms the nested call into a readable linear pipeline:

```
decimal result = 1234.56m
    .Pipe(discount10)
```

```
    .Pipe(fee5)
    .Pipe(round);
```

2.4 Task 4: Dynamic Pipeline Construction with `Aggregate`

When pipeline stages are determined at runtime, functions can be supplied in a collection and folded using `Enumerable.Aggregate`:

```
public static Func<decimal, decimal> Pipeline(params Func<decimal, decimal>
{
    return steps.Aggregate(
        (Func<decimal, decimal>)(x => x),
        (acc, step) => x => step(acc(x))
    );
}
```

Pipeline Configuration	Sequence of Functions	Raw Decimal Output	Explanation
1. Standard	discount10 → fee5 → round	`1116.10`	10% discount on order only; fee added in full afterwards; cents rounded.
2. Swapped	fee5 → discount10 → round	`1115.60`	Fee is added first, so discount also reduces the fee by 10% (0.50 cheaper).
3. Round First	round → discount10 → fee5	`1116.1040`	Rounding executed when input already has 2 decimals; subsequent discount produces 4 decimal places which remain unrounded.
4. Empty Pipeline	Pipeline()	`1234.56`	Returns the identity function; input passes through unaltered.

3. Execution Output

```
Assignment 5 - Composing Functions Using Pipelining
Maxsutkhan Ramazanov | IT-2506

--- Task 1: Currying & Partial Application ---
Initial Amount: 1234,56
After 10% discount: 1111,1040
After 5 flat fee: 1116,1040
After rounding: 1116,10

--- Task 2: Function Composition ---
```

```
Order amount: 1234,56
Checkout result: 1116,10

--- Task 3: Pipelining with Extension Method ---
Order amount: 1234,56
Piped result: 1116,10

--- Task 4: Dynamic Pipeline with Aggregate ---
Input amount: 1234,56
1. Standard (discount -> fee -> round): 1116,10
2. Swapped (fee -> discount -> round):   1115,60
3. Round First (round -> discount -> fee): 1116,1040
4. Empty Pipeline ():                     1234,56
```

4. Answers to Assignment Questions

Question 1: Pure Functions & Payment Systems

Why purity is required: Function composition assumes referential transparency and associativity: (f C g) C h = f C (g C h). If a step contains side effects (e.g. database mutations, network I/O, or clock dependency), reordering, lazy execution, or retrying pipelines becomes non-deterministic and unsafe.

Payment system example: A step `ChargeCreditCard(amount)` would immediately capture funds during pipeline evaluation. Any speculative evaluation, dry run, or reordering would trigger real duplicate charges on customer bank cards.

Question 2: Currying vs Partial Application & Parameter Ordering

Difference: Currying transforms a function of *n* arguments into a chain of *n* unary functions (f(a,b) → a → b → c). Partial application fixes one or more arguments, returning a function accepting the remainder.

Parameter ordering: Configuration parameters (`rate` , `fee`) are placed first, and the data operand (`amount`) last. This allows partially applying the configuration once to yield a clean unary function (`decimal -> decimal`) that flows directly into pipeline steps.

Question 3: Pipeline Ordering Differences & Position of Rounding

Why results differ: Multiplication and addition do not commute: (x − 0.1x) + 5 = 0.9x + 5 ≠ (x + 5) × 0.9 = 0.9x + 4.5. Applying fee first allows the discount to decrease the fee by 0.50.

Rounding first: Rounding an already rounded number at the start produces no effect, but subsequent discount calculation produces fractional cents

(`1116.1040`). Rounding must always be the final step to guarantee currency invariants.

Question 4: Compose vs Pipe vs Pipeline & Identity Seed

Comparison: `Compose` builds reusable functions in point-free style. `Pipe` provides readable left-to-right evaluation on values. `Pipeline` builds dynamic pipelines from runtime collections.

Identity seed: `Pipeline()` with no arguments returns the identity function `x => x` . In `Aggregate` , identity serves as the neutral element of function composition (f ∁ id = id ∁ f = f), guaranteeing safe folding without edge-case branching.